Member:

- 223026679 Janvier NSHIMIYIMANA

COMPUTER ENGINEERING LEVEL_3

GAKO COMPUS

MOBILE APPLICATION

On 26th mar, 26

MOBILE APP LAB5 LOCALSTORAGE IN FLUTTER

This lab focuses on implementing local data storage in a Flutter application using SQLite. The goal is to design a simple posts management system where users can create, view, update, and delete posts without requiring an internet connection. By integrating the sqflite package, the application stores data persistently on the device, ensuring that user-generated content remains available across sessions. This lab also introduces asynchronous database operations and demonstrates how Flutter interacts with local storage efficiently.

After implementing code on that simple posts management system we answered this questions:

1. Which dependencies did you use and why? - Explain why SQLite is necessary to store data locally.

2. How do you handle database exceptions?

• Handle database not initialized

• Handle insert/update/delete errors

• Handle invalid or corrupted data

3. Explain SQLite in Flutter

• Database vs Table

• CRUD operations

• How Flutter interacts with the database asynchronously

Answers:

Name: Jennifer NUTHIMIYIMANA.

Reg No: 22302667

UR-GAKO Campus.

Computer Engineering L3

Mobile Application

On 26th Mar-26

Lab5 Localstorage in flutter.

Q1 Which dependencies did I used and why? - Explain why sqlite is necessary to store data locally?

Dependencies used and why?

- → flutter/material.dart: flutter's UI framework for building the interface.
- → sqflite: used to store data locally in sqlite database.
- → path: used to locate the database file in device storage.

Why SQLite is necessary?:

Sqlite allows app's to store and manage data offline, ensure fast access, and keeps data persistent even after the app is closed.

Q2: Handling database exceptions.

a) Database not initialized.

→ Ensure database is created before use (initialize in main or helper class).

Ex: Database? _database;

```
Future<Database> get database async { if (_database != null) return _database!;
_database = await openDatabase('posts.db', version:1,
onCreate: (db, version) async { await db.execute(
'CREATE TABLE posts (id INTEGER PRIMARY KEY, title TEXT, content
TEXT)'); } );
return _database!;
}
```

→ Use try-catch to catch and handle runtime errors.

ex: ⊗ try {
    await db.insert('posts', post.toMap());
  } catch (e) { print("insert Error : $e"); }

⊗ try { await db.update('posts', post.toMap(),
      where : 'id = ?', whereArgs : [post.id]);
  } catch (e) { print("update error : $e");

⊗ try { await db.delete('posts', where : 'id = ?', whereArgs : [id]);
    } catch (e) { print("Delete Error : $e"); }

- Invalid or corrupted data.
  → Validate input before saving and check retrieved data before displaying.

ex: ⊗ if (title.isEmpty || content.isEmpty) {
      print("Invalid input");
      return;
    }

- this Validate input before saving.

⊗ final data = await db.query('posts');
    if (data.isEmpty) { print("No data found"); }

- Check retrieved data before displaying validated.

Q3: Explain SQLite in flutter.

- Database vs Table.
  database : single file containing multiple tables (posts.db).
  Table : structured collection of related data (Posts table with columns : id, title, content, timestamps, etc...).

- CRUD operations:
  Create : insert() - insert new data with timestamps.
  Read : query() - Retrieves records with ordering and filtering.
  Update : update() - Modifies existing records with change detection.
  Delete : delete() - Removes records with confirmation dialog.

-

- Asynchronous interaction.
  - → Future API : All database operations return Future <T> for non-blocking UI.
  - → async / await : Use dart's async pattern to prevent UI freezing.
  - → Thread isolation : Database operations run on separate thread via platform channels.
  - → state management : setState() or FutureBuilder updates UI after async completion.
  - performance : UI remains responsive while database operations execute in background.

In this lab, a functional offline posts management system was successfully developed using SQLite in Flutter. The application supports full CRUD operations and demonstrates how local databases can be used to maintain persistent data. Additionally, proper handling of database initialization, exceptions, and asynchronous operations ensures smooth performance and reliability. Overall, this lab provides a solid foundation for building offline-capable mobile applications with efficient data management.